Advantech SOP: Using SUSI API to Test and Simulate Watchdog Timer Functionality on Ubuntu Linux

By Larry To

1. Purpose

This SOP provides a step-by-step guide to understand, configure, compile, and execute a program that simulates a system crash using the Advantech SUSI (Secure and Unified Smart Interface) API's Watchdog Timer (WDT). It is written to help individuals with no prior experience in Linux or programming.

The goal is to:

- Understand the **purpose and function of a Watchdog Timer**
- Learn how to **use SUSI API to control the Watchdog Timer**
- Simulate a crash scenario to **verify the system auto-reset behavior**

---

2. Definitions

Watchdog Timer (WDT): A hardware timer that resets the system if the software becomes unresponsive. Used in industrial systems to ensure continued operation in the event of software crashes.

SUSI API: A software library from Advantech that allows access to hardware features on industrial PCs such as GPIO, I2C, thermal sensors, and the watchdog timer.

Triggering the Watchdog: The process of telling the watchdog, "The system is healthy." If you don't trigger it within a certain time, it assumes a crash and resets the system.

Delay Time: The time in milliseconds after starting the WDT before it becomes active.

Reset Time: The time the watchdog waits after becoming active before resetting the system if no trigger occurs.

Event Type: The action the watchdog takes when time expires. Example: event_type = 1 tells watchdog to perform a system reset when timeout occurs.

---

3. System Requirements

- Advantech Industrial PC (e.g., EI-52)
- Pre-installed customized Ubuntu with SUSI API

- Installed build-essential package
- `gcc` compiler

---

4. Step-by-Step Procedure

4.1 Setup Project Directory

Make the directory: mkdir ~/SUSI_training/watchdog_test
Go into your project folder/directory: cd ~/SUSI_training/watchdog_test
Copy SUSI header files into your directory: cp /usr/local/Advantech/SUSI/Susi4Demo/*.h ./

4.2 Create C Source File

Open a new file in a text editor:

```
nano watchdog_test.c
```

Paste the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdint.h>
#include "Susi4.h"

int main() {
    uint32_t status;
    uint32_t delay = 2000;  // Delay period in milliseconds
    uint32_t reset = 3000;  // Time before system resets after delay
    uint32_t event_type = 1; // 1 = RESET

    printf("Initializing SUSI library...\n");
    status = SusiLibInitialize();
    if (status != SUSI_STATUS_SUCCESS) {
        printf("Failed to initialize SUSI: 0x%X\n", status);
        return -1;
    }

    printf("Starting Watchdog...\n");
    status = SusiWDogStart(SUSI_ID_WATCHDOG_1, delay, 0, reset, event_type); //Starts Timer
    if (status != SUSI_STATUS_SUCCESS) {
        printf("Failed to start watchdog: 0x%X\n", status);
        SusiLibUninitialize();
```

```
        return -1;
    }

    printf("System is alive. Triggering watchdog for 10 seconds...\n"); // Simulates normal system
    for (int i = 0; i < 10; i++) {
        SusiWDogTrigger(SUSI_ID_WATCHDOG_1);
        sleep(1);
    }

    printf("Simulating crash. NOT triggering the watchdog...\n"); // Delays timer, restarts system
    sleep((delay + reset) / 1000 + 5);

    printf("System did not reset. Test failed.\n");
    SusiWDogStop(SUSI_ID_WATCHDOG_1);
    SusiLibUninitialize();
    return 0;
}
```

Press `Ctrl+O`, then `Enter`, then `Ctrl+X` to save and exit.

4.3 Create Makefile

Open the Makefile:

```
nano Makefile
```

Paste:

```
TARGET = watchdog_test
SRC = watchdog_test.c
OBJ = $(SRC:.c=.o)

CC = gcc
CFLAGS = -D_LINUX -Wall -Werror -O2 \
  -I/usr/local/Advantech/SUSI/Susi4Demo
LDFLAGS = -L/usr/local/Advantech/SUSI/Driver
LDLIBS = -lSUSI-4.00

all: $(TARGET)
$(TARGET): $(OBJ)
	$(CC) $(CFLAGS) $(OBJ) -o $(TARGET) $(LDFLAGS) $(LDLIBS)

clean:
```

```
	rm -f $(TARGET) *.o
```

Save and exit with `Ctrl+O`, `Enter`, `Ctrl+X`.

4.4 Build the Program

Type make in the terminal

If successful, you will see an executable called `watchdog_test`.

4.5 Run the Program

sudo ./watchdog_test (sudo gives root/admin privileges)

Expected behavior:

- You will see logs showing initialization, 10 seconds of "alive" triggers.
- Then the program stops triggering and simulates a crash.
- If WDT is working, your system **will reboot automatically** within ~5 seconds.
- **NOTE:** Depending on the device, the watchdog timer could trigger immediately based on your system settings (e.g. the bios/system could have a built in watchdog to trigger in 3 seconds or less)

If the system does **not** reset:

- You may have incorrect BIOS settings.
- Or your system may not support watchdog reset.

---

5. Summary and Importance

The Watchdog Timer is a **crucial reliability tool** in embedded and industrial systems. If the software ever hangs or locks up, the WDT ensures that the system will **automatically recover**. This is especially important in:

- IoT Gateways
- Factory automation
- Remote edge computers

By learning to simulate and test the watchdog with SUSI API, developers can:

- Ensure robustness

- Avoid costly downtime
- Automate failure recovery

---

6. Cleanup

To remove build files:

Type make clean in terminal

---

7. Appendix: Variable Glossary

Variable	Type	Description
`status`	uint32_t	Holds return code from SUSI API calls
`delay`	uint32_t	Time (ms) before watchdog starts counting
`reset`	uint32_t	Time (ms) after delay before reset occurs
`event_type`	uint32_t	What action the watchdog takes (1 = RESET)